CA 1 Study Notes

Practical ML CA: method, EDA, preprocessing, pipeline, results, ethics. Beginner level, short, with code you can copy.

What the CA asks for

You get a dataset and 1.5 hours. You must **do** and **justify** six things. Use them as the headings of your notebook:

#	Section	What to show	Time
1	Type of method	Classification, regression or clustering, and why. Which models.	5 min
2	Exploratory analysis	Shape, types, missing, duplicates, target, plots, findings	15 min
3	Preprocessing / feature selection	Cleaning, encoding, scaling, choosing features, and why	15 min
4	ML pipeline	Split, scale, train 2-3 models, cross-validate	20 min
5	Results	Metrics table, confusion matrix / plot, best model, limitations	15 min
6	Ethical concerns	Bias, privacy, transparency, accountability, linked to THIS data	10 min
	Buffer	Fix errors, tidy markdown, re-run all cells	10 min

> **Key idea:** Marks go to the **justification**, not only the code. After every code cell, write 1-3 lines: *what I did, why, what I found*.

Files in this pack (study_pack/code/)

File	Use it for
`make_practice_data.py`	Run once: creates patients.csv, houses.csv, customers.csv
`01_eda.py`	EDA template (change FILE and TARGET)
`02_preprocessing.py`	Cleaning, encoding, feature selection, scaling
`03_classification.py`	Train and compare classifiers, evaluate, fairness check
`04_regression.py`	Train and compare regressors, MAE / RMSE / R2
`05_clustering.py`	KMeans, elbow method, silhouette score
`06_text_classification.py`	Bag of Words / TF-IDF + Naive Bayes / Logistic Regression
`07_CA_answer_template.ipynb`	Notebook with the 6 CA headings: copy it for the real CA

The .py files are split into cells with `# %%`. In VS Code, click *Run Cell* above each one (or press Shift+Enter).

1. Choosing the method

Step 1: look at the target column

The target is...	Task	Example	Start with
A category (yes/no, spam/ham, A/B/C)	**Classification** (supervised)	Heart disease? Loan approved?	Logistic Regression
A number (price, salary, temperature)	**Regression** (supervised)	House price, salary	Linear Regression
Not given, find groups	**Clustering** (unsupervised)	Customer segments	KMeans
Text + a category	**Text classification**	Spam, review sentiment	TF-IDF + Logistic Regression / Naive Bayes

```
df["target"].nunique()   # few values (2-10) -> classification
df["target"].dtype       # float with many values -> regression
```

Step 2: pick 2-3 models to compare

Model	When to use	Good	Watch out	Scale?
Logistic Regression	Classification baseline	Simple, fast, coefficients explain it	Only straight-line boundaries	Yes
Linear Regression	Regression baseline	Simple, coefficients explain it	Only straight-line relationships; outliers hurt	Yes
KNN	Small data	Very simple idea (nearest neighbours)	Slow on big data; must scale	Yes
Decision Tree	When you must explain the rules	Easy to draw and explain	Overfits; limit max_depth	No
Random Forest	Best accuracy on tabular data	Accurate, gives feature importance	Harder to explain	No
Naive Bayes	Text (word counts)	Very fast, good for spam	Assumes features independent	No
KMeans	Clustering	Simple, fast	Must choose k; must scale	Yes

> **Tip:** A safe choice: **one simple, explainable model** (Logistic / Linear Regression) + **one stronger model** (Random Forest). Compare them, then pick one, trading off accuracy against how easy it is to explain.

How to write it

"The target **heart_disease** *has two values (0/1), so this is a supervised* **binary classification** *problem. I compare Logistic Regression (simple, explainable baseline), a Decision Tree (easy-to-read rules) and a Random Forest (usually more accurate). Because this is a medical task, I focus on* **recall** *for class 1: missing a sick patient is the worst error."*

2. Exploratory Data Analysis (EDA)

Goal: understand the data and find problems **before** modelling. Run these in order (full template: `01_eda.py`).

Step	Code	Look for / write
Size and types	`df.shape df.info()`	Rows, columns, numeric vs text columns
Summary stats	`df.describe()`	Strange min/max (age 200, negative price)
Missing values	`df.isnull().sum()`	Which columns, how many: fill or drop?
Duplicates	`df.duplicated().sum()`	Drop them
Target balance	`df[TARGET].value_counts()`	Imbalanced (e.g. 90/10)? Then accuracy misleads, use F1
Distributions	`df.hist()`	Skewed columns, odd values
Outliers	`sns.boxplot(data=df[[...]])`	Real values or data errors?
Correlation	`sns.heatmap(df.corr(numeric_only=True), annot=True)`	Features linked to the target; features linked to each other
Category vs target	`sns.countplot(x=col, hue=TARGET, data=df)`	Does the group change the outcome?

Minimal EDA code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("patients.csv")
print(df.shape); df.info()
print(df.describe())
print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
print(df["heart_disease"].value_counts())

df.hist(figsize=(10, 8)); plt.show()
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm"); plt.show()
```

Example findings (from patients.csv)

- 815 rows, 9 columns: 6 numeric, 3 categorical (sex, smoker, exercise).
- Missing: bmi 52, cholesterol 43, exercise 35. **Plan:** fill (median / mode).
- 15 duplicate rows. **Plan:** drop.
- Target 57% no / 43% yes: fairly balanced, but I still report F1 and recall.
- blood_pressure has values of 300: impossible, so data-entry errors. **Plan:** remove.
- Highest correlation with the target: age (0.25), blood_pressure (0.23), cholesterol (0.21).

> **Tip:** End EDA with a list of findings, each one followed by its **plan**. That plan is your justification for the preprocessing section.

3. Preprocessing and feature selection

Problem -> fix -> code

Problem	Fix (and why)	Code
Duplicate rows	Drop: they count the same row twice	`df = df.drop_duplicates()`
ID / name columns	Drop: there is no pattern to learn	`df = df.drop(columns=['id'])`
Missing numbers	Median: not pulled by outliers	`df[c] = df[c].fillna(df[c].median())`
Missing categories	Mode (most common value)	`df[c] = df[c].fillna(df[c].mode()[0])`
Very few missing rows	Just drop them	`df = df.dropna()`
Impossible values	Remove those rows	`df = df[df['bp'] < 250]`
Yes/no text	Map to 0/1	`df[c] = df[c].map({'no':0,'yes':1})`
Category, 3+ values	One-hot encoding	`pd.get_dummies(df, columns=[c], drop_first=True, dtype=int)`
Different ranges	StandardScaler (mean 0, std 1). Needed by LogReg, KNN, KMeans	`scaler.fit_transform(X_train)`
Text column	CountVectorizer / TfidfVectorizer	`TfidfVectorizer(stop_words='english')`

Rule: Data leakage: split into train/test **first**, then `fit` the scaler or vectorizer on **train only** and just `transform` the test set. Otherwise the model has "seen" the test data and the score looks too good.

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)   # learn mean/std from train
X_test  = scaler.transform(X_test)        # reuse them on test
```

Feature selection: 3 simple methods

Method	Code	Keep
Correlation with target	`df.corr()[TARGET].sort_values()`	Features clearly + or -, not near 0
Random Forest importance	`rf.fit(X_train, y_train)` `pd.Series(rf.feature_importances_, index=X.columns)`	Top features
Common sense / ethics	`df.drop(columns=['id', 'name'])`	Drop IDs and leaky columns (known only after the outcome). Think twice about sensitive ones (sex, race)

Example: *"I kept age, blood_pressure, cholesterol, smoker and bmi because they had the highest correlation and importance. sex had near-zero correlation, so I dropped it as a feature but kept it aside to check fairness."*

4. The ML pipeline

Load -> EDA -> Clean -> Encode -> Split -> Scale -> Train -> Cross-validate -> Evaluate on test -> Explain -> Ethics

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

X = df.drop(columns=["heart_disease"])
y = df["heart_disease"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)   # no stratify for regression

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

models = {"Logistic Regression": LogisticRegression(),
          "Random Forest": RandomForestClassifier(random_state=42)}
for name, model in models.items():
    model.fit(X_train, y_train)                                    # train
    pred = model.predict(X_test)                                   # predict
    cv = cross_val_score(model, X_train, y_train, cv=5).mean()   # 5-fold CV
    print(name, "CV:", round(cv, 3), "Test acc:", round(accuracy_score(y_test, pred), 3),
          "F1:", round(f1_score(y_test, pred), 3))
```

Key terms to explain

Term	Meaning in one line
Train / test split	Learn on 80%, check on the unseen 20%, so we measure performance on new data
random_state=42	Same split every time, so results can be reproduced
stratify=y	Train and test keep the same class ratio (classification)
Cross-validation (cv=5)	Train/test on 5 different splits and average: more reliable than one split
Baseline	Always guess the most common class (or the mean). Your model must beat it
Overfitting	Train score much higher than test score: the model memorised. Fix: simpler model, max_depth
Underfitting	Both scores low: the model is too simple or the features are weak
Hyperparameter	A setting you choose before training (max_depth, n_neighbors, k). Try a few values

```
for depth in [2, 4, 6, None]:          # simple tuning + overfitting check
    tree = DecisionTreeClassifier(max_depth=depth, random_state=42).fit(X_train, y_train)
    print(depth, tree.score(X_train, y_train), tree.score(X_test, y_test))
```

5. Evaluation and results

Classification

Metric	Meaning	Use when
Accuracy	% of all predictions that are correct	Classes are balanced
Precision	Of the ones predicted positive, how many really are	False alarms are costly (spam filter)
Recall	Of the real positives, how many we found	Misses are costly (disease, fraud)
F1-score	Balance of precision and recall	Imbalanced classes; one number to compare
Confusion matrix	Table of TN, FP, FN, TP	Always show it: it tells you **which** errors happen

```
from sklearn.metrics import classification_report, ConfusionMatrixDisplay
print(classification_report(y_test, pred))
ConfusionMatrixDisplay.from_predictions(y_test, pred); plt.show()
```

Reading the confusion matrix (rows = actual, columns = predicted):

	Predicted 0	Predicted 1
Actual 0	TN (correct)	FP: false alarm
Actual 1	FN: **missed case**	TP (correct)

Regression

Metric	Meaning	Good value
MAE	Average size of the error, in target units (e.g. EUR 21,000)	Lower
RMSE	Like MAE, but big errors count more	Lower
R2	Share of the variation explained (1 = perfect, 0 = no better than guessing the mean)	Close to 1

Clustering

- **Elbow plot** (inertia vs k): pick the k where the curve bends.
- **Silhouette score** (-1 to 1): higher means better-separated clusters.
- Describe each cluster with `df.groupby('cluster').mean()` and give it a name.

Writing the results: example from 03_classification.py

Model	Test accuracy	F1 (class 1)
Baseline (always 0)	0.572	-
Logistic Regression	**0.692**	**0.632**
KNN (k=5)	0.635	0.586
Decision Tree (depth 4)	0.610	0.523
Random Forest	0.591	0.480

"Logistic Regression performed best (accuracy 0.69, F1 0.63, 5-fold CV 0.70), beating the 0.57 baseline. Recall for heart disease is 0.62, so about 38% of sick patients are missed: too many for real medical use. The strongest features were cholesterol, age and blood pressure. Limitations: small synthetic data, few features, little tuning."
Note that the simplest model won here: a complex model is not automatically better.

6. Ethical concerns

Always link ethics to **this** dataset and **this** use. A generic list scores low; a specific risk with a mitigation scores high.

Concern	What it means	Example	What to do
Bias / fairness	Model works worse for, or is harsher on, a group	Loan model approves fewer women because past approvals were biased	Compare metrics per group; balance the data; human review
Proxy variables	A 'neutral' feature stands in for a sensitive one	Postcode can stand in for ethnicity or income	Check correlations with sensitive columns; remove or justify
Privacy	Personal or sensitive data (health, finance)	Patient records with names / IDs	Remove IDs, anonymise, consent, GDPR, collect only what is needed
Transparency	People can't see why the model decided	Rejected applicant gets no reason	Prefer explainable models; show feature importance
Accountability	Who is responsible for a wrong decision?	Model misses a disease	Human in the loop; the model only supports the decision
Error cost	Some errors hurt people more	False negative in cancer screening	Choose the metric (recall) and threshold with this in mind
Data quality	Old, small or unrepresentative data	Trained only on one hospital / one country	State limitations; collect more diverse data

Simple fairness check (in 03_classification.py)

```
test = X_test.copy()           # X_test before scaling (a DataFrame)
test["actual"] = y_test
test["pred"] = pred
print(test.groupby("sex_M")[["actual", "pred"]].mean())
# per group: real positive rate vs predicted positive rate
# (03_classification.py also prints accuracy per group)
```

Result on patients.csv: **female** accuracy 0.65, predicted positive rate 0.35; **male** accuracy 0.73, predicted positive rate 0.46. The model is less accurate for women, so it may miss more female patients. That is a fairness risk to report and fix.

Template paragraph

"This model uses health data, which is sensitive, so IDs were removed and the data must be stored under GDPR with patient consent. The model is less accurate for women (0.65 vs 0.73), possibly because there are fewer women in the data, so it could miss female patients more often. Its recall of 0.62 means many sick patients are missed, so it should only **support** *a doctor, never replace one. I chose Logistic Regression partly because its coefficients can be explained to patients and doctors. Next steps: collect more balanced data, check performance per group, and lower the decision threshold to improve recall."*

7. Practice tasks

Do them in order. Tick each one off. The answers come from running the code files.

A. Learn the steps (with the practice data)

[ ] **1.** Run `make_practice_data.py`. Open patients.csv and say in words what each column is.

[ ] **2.** Run `01_eda.py` cell by cell. Write 5 EDA findings, each with a plan.

[ ] **3.** In 02, change missing-value filling to `df.dropna()`. How many rows are left? Which is better here, and why?

[ ] **4.** In 02, find the 3 most important features by correlation and by Random Forest. Do they agree?

[ ] **5.** In 03, which model wins? Is it better than the baseline? Write the results table.

[ ] **6.** In 03, read the confusion matrix: how many sick patients were missed (FN)?

[ ] **7.** In 03, change the Decision Tree max_depth. At which depth does it start to overfit?

[ ] **8.** In 03, run the fairness cell. Write a 4-line ethics paragraph about the gap you see.

[ ] **9.** Run 04 (regression). Explain MAE in plain words ("on average we are off by ...").

[ ] **10.** Run 05 (clustering). Use the elbow and silhouette to choose k, then name each cluster.

[ ] **11.** Run 06 (text). Accuracy is 0.97, but what is the recall for spam? Why does that matter?

B. Mock CAs: 90 minutes each, use 07_CA_answer_template.ipynb

Mock	Dataset	Task	Things to think about
1	sklearn `load_breast_cancer(as_frame=True).frame`	Classification (target)	Which error is worse? Recall vs precision. Patient privacy
2	experiments/linear_regression/Housing.csv	Regression (Price)	Drop the Address text column. Could 'Area Income' make prices unfair for poorer areas?
3	experiments/movie_review_classifier/movie_reviews(in).csv	Text classification	BoW vs TF-IDF. Bias in language, e.g. slang or dialect
4	customers.csv (this pack)	Clustering	Choose k, name the segments. Is it fair to target ads at 'low income' groups?

Exam habit: In each mock, write all six headings first, then fill them in. Leave 10 minutes at the end to run **Restart & Run All** so the notebook works from top to bottom.

8. One-page cheat sheet

```
# ---- imports ----
import pandas as pd, matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.cluster import KMeans
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import (accuracy_score, f1_score, classification_report,
    ConfusionMatrixDisplay, mean_absolute_error, mean_squared_error, r2_score, silhouette_score)

# ---- EDA ----
df.shape; df.info(); df.describe(); df.isnull().sum(); df.duplicated().sum()
df[T].value_counts(); df.hist(); sns.heatmap(df.corr(numeric_only=True), annot=True)

# ---- clean ----
df = df.drop_duplicates().drop(columns=["id"])
df[c] = df[c].fillna(df[c].median())          # numbers
df[c] = df[c].fillna(df[c].mode()[0])         # categories
df = pd.get_dummies(df, columns=[c], drop_first=True, dtype=int)

# ---- split + scale ----
X = df.drop(columns=[T]); y = df[T]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
sc = StandardScaler(); X_train = sc.fit_transform(X_train); X_test = sc.transform(X_test)

# ---- train + evaluate (classification) ----
m = LogisticRegression().fit(X_train, y_train); pred = m.predict(X_test)
print(classification_report(y_test, pred)); ConfusionMatrixDisplay.from_predictions(y_test, pred)
cross_val_score(m, X_train, y_train, cv=5).mean()

# ---- regression ----
mean_absolute_error(y_test, pred); mean_squared_error(y_test, pred) ** 0.5; r2_score(y_test, pred)

# ---- clustering ----
labels = KMeans(n_clusters=k, n_init=10, random_state=42).fit_predict(X); silhouette_score(X, labels)

# ---- text ----
tf = TfidfVectorizer(stop_words="english")
Xtr = tf.fit_transform(text_train); Xte = tf.transform(text_test)
```

Quick decisions

If...	Then...
Target is a category / a number / missing	Classification / Regression / Clustering
Classes are imbalanced	Use stratify=y; report F1 and recall, not just accuracy
Missing values in a column	Few: dropna. Some: median / mode. Most (>50%): drop the column
Model is LogReg, LinReg, KNN, KMeans	Scale the features
Train score much higher than test	Overfitting: simpler model, lower max_depth
Health, money, jobs, people	Write about bias, privacy, transparency, human oversight